Engineering Decisions Report: Retrieval Layer & Grounded RAG Assistant

Project: Data Pipeline & Agentic AI Traffic Accident Processing System

Author: Liza Golan

1. Executive Summary & Introduction

This document outlines the engineering decisions, baseline metrics, failure analysis, and optimization iterations for the Retrieval-Augmented Generation (RAG) assistant built as part of the Data Science module 5 exercise.

The primary objective of this module was to construct a deterministic, reliable RAG pipeline over a scientific arXiv corpus to maximize factual grounding, eliminate hallucinations entirely, and handle out-of-scope queries gracefully through strict negative constraints.

Through iterative architectural refinement—transitioning from baseline vector search ($k = 5$) to a wide retrieval with keyword-boosted reranking ($k = 20 \rightarrow Top5$)—the system successfully improved its groundedness score from a baseline of **0.267** to an optimized score of **0.400**, achieving a positive performance lift of **+0.133**.

2. Engineering Approach & Justification

A. Chunking Strategy

- **Decision:** Whole-Text Abstract Documents (No secondary splitting).
- **Justification:** arXiv article abstracts are naturally concise and self-contained (ranging from 80 to 150 words), each encapsulating a distinct scientific problem and core solution. Splitting such brief text blocks into arbitrary sub-sentences would fracture conceptual coherence, introduce lexical noise during retrieval, and degrade semantic embedding alignment.

B. Embedding & Vector Indexing

- **Decision:** Local `all-MiniLM-L6-v2` model via `Sentence Transformers`, indexed within a local `Chroma DB` vector store.
- **Justification:** This model provides an optimal balance between low computational latency and high semantic accuracy (Normalized Cosine Similarity) across scientific and technical text. Indexing was accompanied by rich metadata (`paper_id`, `category`, `year`) to enable structured filtering when necessary.

C. System Prompt & Negative Constraint Enforcement

- **Decision:** Strict Role-Prompting combined with a hard negative constraint.
- **Justification:** To eliminate hallucinations—the most critical failure mode in RAG systems—the model was explicitly instructed to disregard prior parametric knowledge, rely exclusively on the provided context, and return a deterministic refusal string (`"I don't know"`) whenever supporting evidence is missing.

D. Metric Refinement & Evaluation Bugfixing (arXiv ID Regex)

- **The Challenge:** During initial evaluation, the automated groundedness checker was returning false zeros (0.0) even when the model cited the right paper. It was discovered that there is a parsing bug: the evaluation script expected pure arXiv IDs, but the text contained version tags (e.g., `2301.12345v1`).
- **The Fix :** Implementing a robust regular expression (`r"\d{4}\.\d{4,5}(?:v\d+)?"`) to properly normalize and extract versioned arXiv IDs during metric calculation, ensuring our baseline measurements were mathematically accurate and fair.

3. Failure Mode Analysis & Grounding Verification

In order to evaluate the pipeline, a frozen evaluation set of 20 questions was established: 15 in-corpus questions and 5 completely out-of-scope queries from unrelated domains.

- **Out-of-Scope Test (Refusal Benchmark):** The system achieved a **100% success rate**, successfully refusing all 5 out-of-corpus questions. The model correctly identified the semantic gap and refrained from fabricating answers.
- **Baseline Groundedness Score:** Evaluated across the 15 in-corpus questions, the initial baseline groundedness score was **0.267**.
- **Baseline Failure Analysis:** A score of 0.267 indicated that pure semantic vector search ($k = 5$) frequently ranked the exact target paper too low or missed precise keyword alignments, causing the model to issue safe refusals for several questions where the correct context was buried outside the top 5 results.

4. Architectural Improvement & The Lift

To improve the baseline groundedness score (0.267), i have tested the assignment's improvement strategies. While system prompt successfully forced the model to refuse unanswerable questions, the main bottleneck was **retrieval recall**—the correct paper wasn't always appearing in the top 5 vector results.

To solve this, implemented **Reranking (Retrieve Wide, Rank Narrow)**:

- **How it was done:** Instead of fetching just $k = 5$, pulled a wider pool of 20 candidate chunks ($k = 20$). Then, wrote a custom keyword-overlap filter that scanned the chunks for key terms from the user's question, re-ordering them so the most relevant context floated straight to the top.
- **Engineering Constraint:** Running evaluations over 15 questions against the Groq API introduced rate-limiting challenges (HTTP 429 errors). We solved this by building robust retry loops and controlled time.sleep(6.0) intervals into our evaluation script.

Official Results (Section 10):

- **before :** 0.267
- **after :** 0.400
- **lift :** +0.133

What this tells us:

By expanding the retrieval breadth and adding keyword reranking, we successfully lifted our groundedness score by $+ \ 0.133 \ [from \ 0.267 \ to \ 0.4]$. It proves that fine-tuning how chunks are ranked before hitting the LLM has a direct, measurable impact on factual accuracy.

5. Conclusion & Takeaways

Following this exercise, below you can find conclusions made:

- **Data & Metric Validation Matter:** Fixing the arXiv ID regex taught me that building evaluation metrics requires handling messy, real-world data formats rather than assuming clean inputs.
- **Retrieval is Usually the Bottleneck:** Simply tweaking prompts wasn't enough; true performance gains came from improving how documents were retrieved and reranked ($k = 20 \rightarrow Top5$).
- **Production Readiness:** Implementing safe refusals, managing API rate limits with retry logic, and tracking a clear "before/after" lift gave me hands-on experience in building robust, trustworthy LLM applications.
- **Baseline vs. Optimization:** Starting with a simple vector search gave a baseline score of 0.267. Moving to a wide-retrieval and reranking strategy helped to understand how adjusting retrieval parameters directly impacts groundedness.
- **The Importance of Grounding:** Enforcing strict negative constraints and checking citations via code keeps the model honest—ensuring it refuses to answer when the data isn't there instead of making things up.